Ex2. Class instantiation

Given an Ontology

```
:Parent owl:equivalentClass [
   owl:intersectionOf ( :Person
      [ a owl:Restriction ; owl:onProperty :hasChild ;
        owl:someValuesFrom :Person ] ) ] .
:Mary a :Person ; :hasChild :Tom .
```

a) Can the reasoner derive that Mary is a Parent?

b) If we add ":Tom a :Person .", does the result change? Explain the difference.

Description Logic:

$$Parent \equiv Person \sqcap \exists hasChild.Person$$

$$Person(Mary), hasChild(Mary, Tom)$$

➔ Split Class equivalence into:

$$Person \sqcap \exists hasChild.Person \sqsubseteq Parent \qquad (1)$$

$$Parent \sqsubseteq Person \sqcap \exists hasChild.Person \qquad (2)$$

NNF:

Axiom (1) : $\neg(\text{Person} \sqcap \exists \text{hasChild}.\,\text{Person}) \sqcup \text{Parent}$

$= \neg\text{Person} \sqcup \neg\exists \text{hasChild}.\,\text{Person} \sqcup \text{Parent}$

$= \neg Person \sqcup \forall hasChild.\neg Person \sqcup Parent$

Axiom (2) : $\neg Parent \sqcup (Person \sqcap \exists hasChild.Person)$

a, Check Class instantiation: Parent (Mary) ?

Add $\neg Parent\ (Mary)$ to the KB and check Consistency for contradiction

Tableau for DL-NNF:

a, $\neg Person \sqcup \forall hasChild.\neg Person \sqcup Parent$
b, $\neg Parent \sqcup (Person \sqcap \exists hasChild.Person)$
c, $Person(Mary), hasChild(Mary, Tom)$
d, $\neg Parent\ (Mary)$

1c, 1d

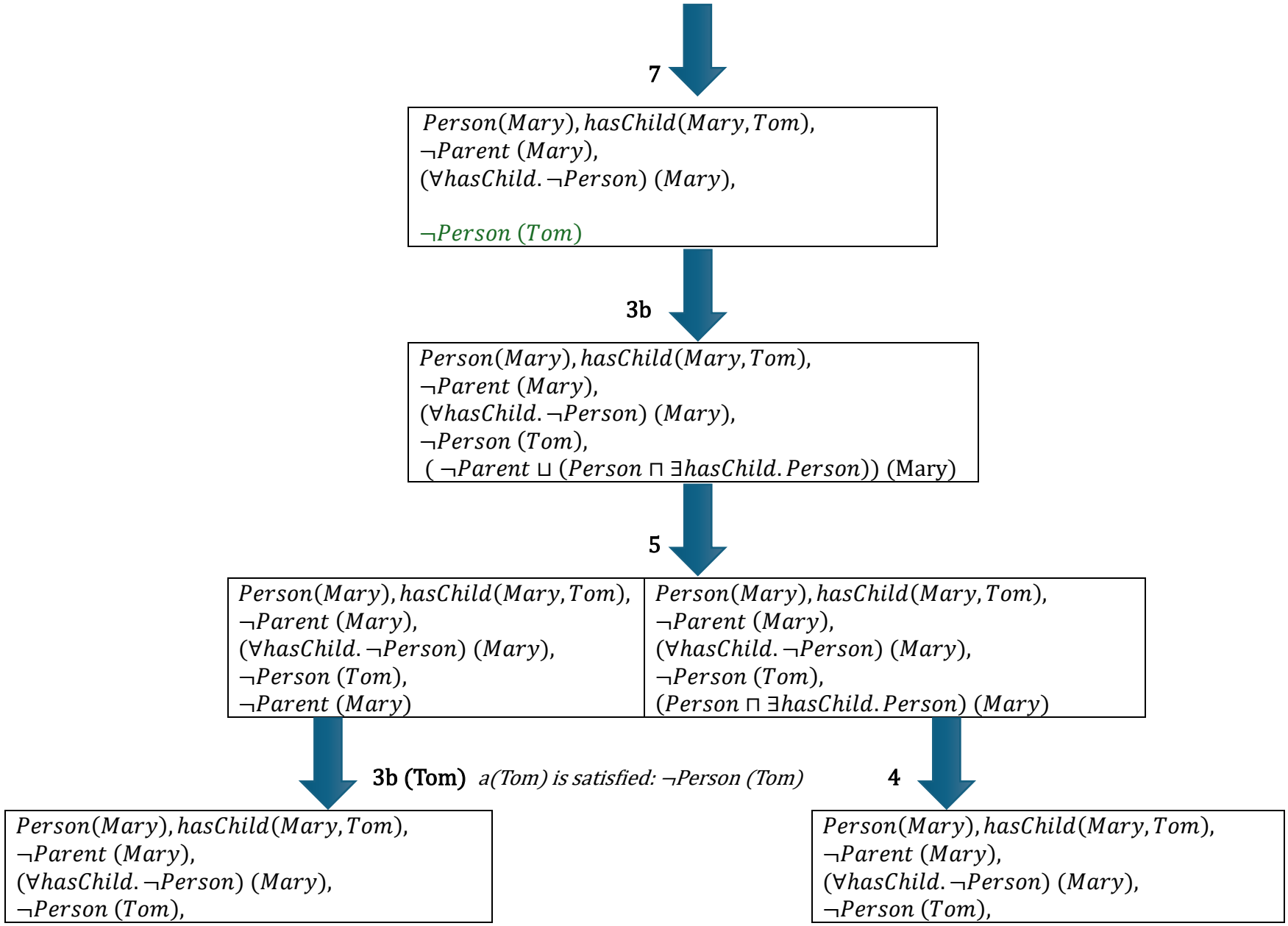
$Person(Mary), hasChild(Mary, Tom),$
$\neg Parent\ (Mary)$

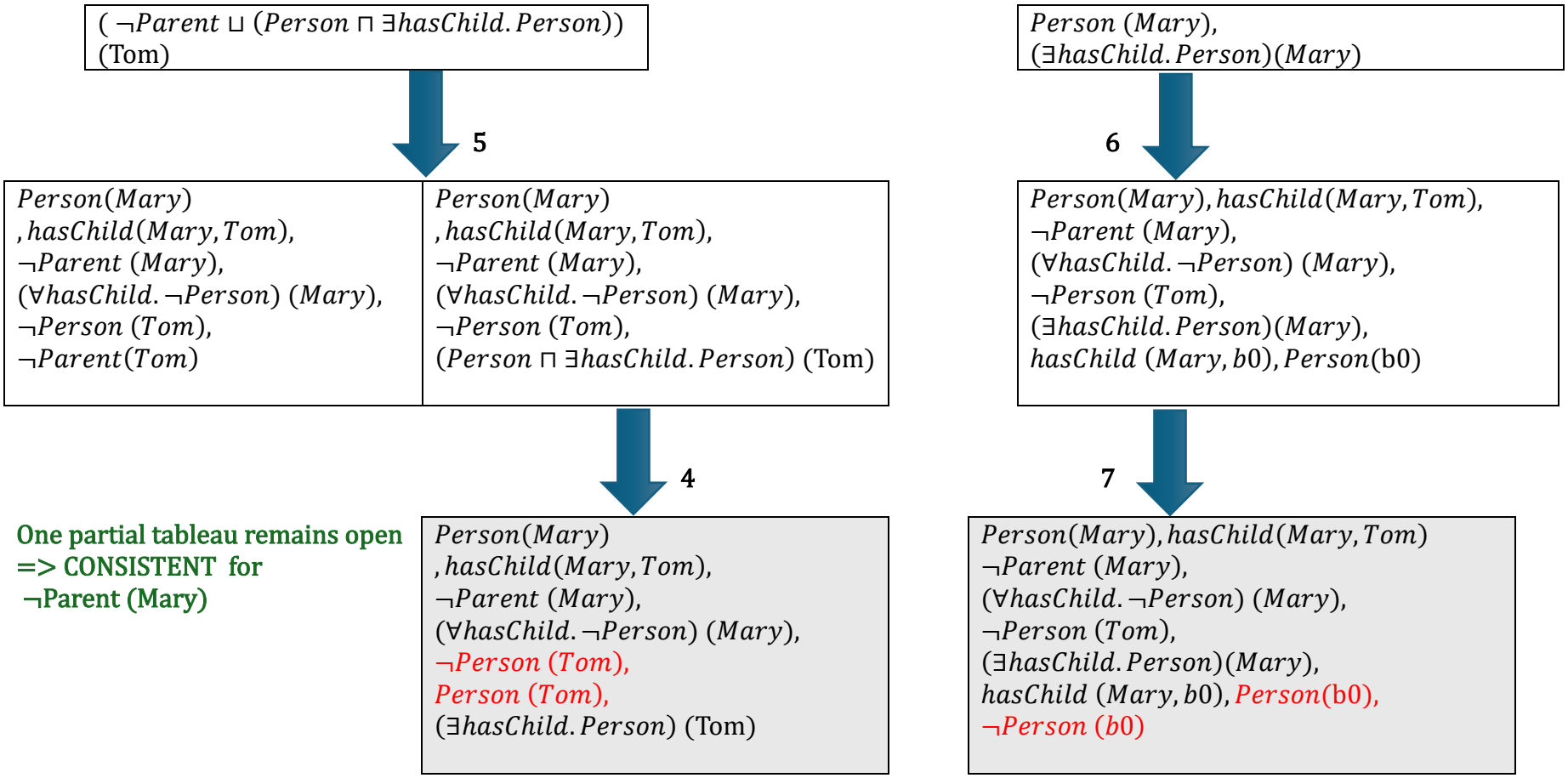
3a

$Person(Mary), hasChild(Mary, Tom),$
$\neg Parent\ (Mary),$
$(\neg Person \sqcup \forall hasChild.\neg Person \sqcup Parent)(Mary)$

5

$Person(Mary),$ $hasChild(Mary, Tom),$ $\neg Parent\ (Mary),$ $\neg Person\ (Mary)$	$Person(Mary),$ $hasChild(Mary, Tom),$ $\neg Parent\ (Mary),$ $(\forall hasChild.\neg Person)\ (Mary)$	$Person(Mary),$ $hasChild(Mary, Tom),$ $\neg Parent\ (Mary),$ $Parent\ (Mary)$


7
Person(Mary), hasChild(Mary, Tom),
¬Parent (Mary),
(∀hasChild. ¬Person) (Mary),
¬Person (Tom)
3b
Person(Mary), hasChild(Mary, Tom),
¬Parent (Mary),
(∀hasChild. ¬Person) (Mary),
¬Person (Tom),
(¬Parent ⊔ (Person ⊓ ∃hasChild. Person)) (Mary)
5
Person(Mary), hasChild(Mary, Tom),
¬Parent (Mary),
(∀hasChild. ¬Person) (Mary),
¬Person (Tom),
¬Parent (Mary)
Person(Mary), hasChild(Mary, Tom),
¬Parent (Mary),
(∀hasChild. ¬Person) (Mary),
¬Person (Tom),
(Person ⊓ ∃hasChild. Person) (Mary)
3b (Tom)
a(Tom) is satisfied: ¬Person (Tom)
4
Person(Mary), hasChild(Mary, Tom),
¬Parent (Mary),
(∀hasChild. ¬Person) (Mary),
¬Person (Tom),
Person(Mary), hasChild(Mary, Tom),
¬Parent (Mary),
(∀hasChild. ¬Person) (Mary),
¬Person (Tom),



=> There's not enough information to derive that Mary is a Parent.

b, Add $Person\ (Tom)$ -> result change?

Based on the output of a, we add $Person\ (Tom)$ to the open branch:

$Person(Mary), hasChild(Mary, Tom)$
$\neg Parent\ (Mary)$,
$(\forall hasChild.\neg Person)\ (Mary)$,
$\neg Person\ (Tom)$,
$\neg Parent\ (Tom)$,
$Person\ (Tom)$

⇨ **All partial tableaus are Closed -> INCONSISTENT**

=> If we add ":Tom a :Person .", the reasoner can derive that Mary is a Parent.